

初中信息技术入门教程：学习导引与总路线

本教程面向初中零基础或基础较弱的学生。学习目标不是一开始就背语法、赶进度，而是一步一步做到：会使用计算机、会管理文件、能理解数据和网络、能把问题拆成步骤、能写简单程序、能测试和修改错误、能完成一个小作品。

你可以把这套教程当作一条学习路线，也可以把每一篇当作一节或几节课的学习材料。

一、这节课要学什么

信息技术学习可以分成六个层次：

- 会使用工具
 - 会管理文件
 - 理解数据和网络
 - 学会计算思维
 - 学习编程基础
 - 完成综合项目

对应到具体内容：

学习层次	要学会什么	为什么重要
计算机基础	硬件、软件、操作系统、窗口、应用程序	知道程序运行在什么环境里
文件管理	文件、文件夹、扩展名、路径、压缩、备份	代码、图片、网页、作业都需要正确保存和提交
键盘输入	英文符号、快捷键、缩进、保存	编程对符号和格式要求很严格
数据与网络	二进制、位、字节、文件大小、浏览器、网址、服务器	理解计算机如何保存和传输信息
计算思维	分解、抽象、算法、流程图、IPO 模型	写代码前先想清楚步骤
图形化编程	事件、顺序、分支、循环、变量	不被语法卡住，先理解程序结构
Python 编程	输入输出、变量、条件、循环、字符串、列表、函数	掌握一门适合入门的文本编程语言
网页或硬件拓展	HTML/CSS/JavaScript 或 micro:bit/机器人	体验程序在网页和现实设备中的应用
综合项目	选题、分析、设计、实现、测试、展示	完整经历一次作品开发过程

二、推荐学习顺序

建议按下面顺序学习，不要一上来就跳到复制代码。

1. 认识计算机系统
2. 文件管理、路径、键盘输入
3. 数据、网络、信息检索与安全
4. 计算思维、流程图与 IPO 模型
5. 图形化编程入门
6. Python 基础编程
7. Python 小项目实践
8. 网页制作或硬件编程拓展
9. 综合项目与展示
10. 自查、评价和改进

如果时间很少，至少完成：

- 文件管理与键盘输入
- 计算思维与流程图
- 图形化编程
- Python 输入输出、变量、条件、循环
- 一个小项目

三、每次学习都按五步走

以后无论学习文件管理、流程图、Scratch、Python 还是网页制作，都按这个顺序做：

- 看懂任务
- 写出步骤
- 动手完成
- 检查测试
- 记录总结

第 1 步：看懂任务

先回答三个问题：

1. 我要解决什么问题？
2. 最后要交什么作品？
3. 需要用到哪些文件、工具或数据？

例如任务是“写一个成绩等级判断程序”，你要先知道：

- 输入：一个分数。
- 处理：根据分数判断等级。
- 输出：优秀、良好、合格或需要努力。

第 2 步：写出步骤

不要急着打开软件。先用自然语言写步骤。

示例：

1. 输入分数
2. 如果分数大于等于 90，输出“优秀”
3. 否则如果分数大于等于 80，输出“良好”
4. 否则如果分数大于等于 60，输出“合格”
5. 否则输出“需要努力”

第 3 步：动手完成

动手时要注意：

1. 文件保存到正确文件夹。
2. 文件名清楚，能看出内容。
3. 代码或作品每完成一小步就保存一次。
4. 不要一次写太多，先做最小版本。

第 4 步：检查测试

至少测试三种情况：

测试类型	示例	目的
正常数据	85 分	看基本功能是否正确
边界数据	60 分、90 分	看分界线是否正确
特殊数据	0 分、100 分	看极端情况是否正确

第 5 步：记录总结

每次学习后写 3 到 5 句话：

1. 今天完成了什么？
2. 用到了哪些概念？
3. 遇到什么错误？
4. 我是怎么解决的？
5. 下次还想改进什么？

四、学习前先建好文件夹

在电脑上建立一个总文件夹：

信息科技学习

在里面建立这些子文件夹：

```
信息科技学习
├── 课堂练习
├── 作业提交
├── 资料图片
├── 程序代码
│   ├── 图形化编程
│   ├── Python练习
│   └── 网页作品
├── 综合项目
└── 学习记录
```

每次上课或自学时，都把文件放在对应位置。不要把代码、图片、文档全部堆在桌面上。

文件命名建议

不推荐

推荐

新建文档.txt 2026-信息科技-学习记录.txt

代码.py Python练习-成绩等级判断.py

最终版2.html 网页作品-校园文明宣传-v2.html

图片1.png 校园照片-操场.png

五、每个作品都要有这四样东西

无论是 Python 程序、网页作品、图形化作品还是硬件项目，都尽量保留下面四样内容。

作品文件
说明文档
测试记录
修改记录

1. 作品文件

例如：

- main.py
- index.html
- Scratch 作品文件
- MakeCode 项目链接或导出文件

2. 说明文档

说明文档可以很短，但必须说清楚：

项目名称：成绩等级判断器
项目功能：输入成绩，输出等级
运行方法：打开 `main.py`，点击运行，输入 0 到 100 的分数
用到知识：输入、数字转换、条件判断

3. 测试记录

示例：

测试数据 预期结果 实际结果 是否通过

95	优秀	优秀	通过
60	合格	合格	通过
59	需要努力	需要努力	通过

4. 修改记录

示例：

问题	原因	修改方法
输入 90 显示良好	判断顺序写错	先判断 90，再判断 80
程序提示 <code>SyntaxError</code>	冒号漏写	在 if 条件后补上英文冒号

六、合理使用网络资料和 AI 工具

网络和 AI 可以帮助学习，但不能代替学习。

可以这样用

1. 搜索概念解释。
2. 查找错误提示的含义。
3. 请 AI 用更简单的话解释一段代码。
4. 让 AI 帮你设计测试数据。
5. 对比自己的思路有没有遗漏。

不应该这样用

1. 直接复制别人或 AI 生成的大段代码当作自己的作品。
2. 不读懂代码就提交。
3. 把真实姓名、电话、住址、账号密码发给陌生网站或 AI。
4. 使用未经允许的图片、音乐、文字、代码却不说明来源。
5. AI 说什么都相信，不检查结果。

使用 AI 后要能回答

如果你用 AI 辅助完成了作品，至少要能回答：

1. 这段代码的输入是什么？
 2. 这段代码的输出是什么？
 3. 变量分别保存什么数据？
 4. 条件判断在判断什么？
 5. 如果运行错误，你会先检查哪里？
-

七、课时安排建议

16 课时快速入门

阶段	课时	内容
编程前准备	2	文件管理、键盘输入、流程图、IPO 模型
图形化编程	4	事件、顺序、条件、循环、变量
Python 基础	8	输出、输入、变量、条件、循环、列表、函数、调试
综合项目	2	完成一个小程序或图形化作品

30 课时标准入门

阶段	课时	内容
计算机基础	6	计算机系统、文件、路径、键盘、数据、网络
计算思维	4	分解、抽象、流程图、IPO、调试
图形化编程	6	动画、游戏、变量、判断、循环
Python 基础	10	输入输出、变量、条件、循环、字符串、列表、函数
综合项目	4	选题、实现、测试、展示

40 课时以上拓展

可以在标准入门后增加：

1. 网页制作：HTML、CSS、JavaScript。
 2. 硬件编程：micro:bit、Arduino、机器人。
 3. Python 深入：文件读写、数据统计、Turtle 绘图。
 4. 信息学竞赛基础：枚举、模拟、排序、简单搜索。
 5. 更完整的项目展示和作品迭代。
-

八、学完本教程后应达到的状态

你不需要变成“编程高手”，但应该做到：

1. 能独立建立项目文件夹并规范保存文件。
2. 能输入常见英文符号，能区分中英文标点。
3. 知道计算机由硬件、软件、数据和网络组成。
4. 知道数据可以用数字表示，理解文件大小的大致含义。
5. 能用流程图或自然语言描述一个任务的步骤。
6. 能说清程序的输入、处理和输出。
7. 能写包含变量、条件、循环的简单程序。
8. 能读错误提示，尝试定位和修改问题。
9. 能用多组数据测试程序。
10. 能完成一个小项目，并说明自己的思路和不足。

最重要的是：以后遇到新问题时，不是只问“答案是什么”，而是先问自己：

我能不能把它拆小？

输入是什么？

处理步骤是什么？

输出是什么？

我怎么验证它对不对？